

Nebula Engine (NE)

A private, C++-only reality engine designed as a deterministic physics + visuals kernel for ANM.

Version: v0.1 (foundation) | Date: 21 Dec 2025

Primary goals	Realistic physics and visuals, tuned to look like reality; range from ant-scale to supernova-scale
Performance target	Locked 60 FPS simulation loop (fixed timestep) with stable frame pacing
Language	C++ (C++20). Platform bindings are optional and separate from core.
Determinism	Engine core must be reproducible for the same seed, inputs, and timestep schedule.
Rendering	Ray tracing as a long-term feature; start with clean renderer interface, then implement.

Contents

- 1 1. Why NE exists
- 2 2. Design rules (non-negotiables)
- 3 3. High-level architecture
- 4 4. Source tree and module ownership
- 5 5. Core loop: time, determinism, 60 Hz physics
- 6 6. ECS: entities and components
- 7 7. Physics: forces, integration, collisions
- 8 8. Rendering: interfaces first, implementations later
- 9 9. ANM bridge: how ANM will interact with NE
- 10 10. Build and run (macOS, clang++)
- 11 11. Cursor workflow tips
- 12 12. Roadmap (ant to supernova)

1. Why NE exists

Nebula Engine is the reality layer for ANM: a clean, deterministic simulation kernel that can be driven by an external intelligence. The core engine stays separate from ANM so that physics and rendering stay correct, testable, and reproducible.

NE is built to scale from tiny scenes (an ant walking) to extreme scenes (stellar dynamics). That scale is managed through units, timestep policy, and world scaling rather than hacks.

2. Design rules (non-negotiables)

- C++-only core. Platform code (Metal, windowing, input) is optional and kept outside core modules.
- Fixed timestep simulation for physics (e.g., 60 Hz) with an accumulator and step budget.
- Determinism: same seed + same inputs + same dt schedule -> same results (within floating point limits).
- Separation: physics cannot depend on rendering; rendering reads world state only.
- Tests first for core math, time stepping, collisions, and determinism.

3. High-level architecture

Layer	Responsibility	Notes
Core	Types, math, time system, world ticking	No platform code here
ECS	Entity registry + component storage	Small and fast; contiguous storage where possible
Physics	Forces, integration, collisions	Stable, deterministic, unit-consistent
Render	Renderer interface + draw calls	Implementation can be Metal/Vulkan later
ANM Bridge	Perception + action surface for ANM	Think: observe state, propose forces/actions

4. Source tree and module ownership

Reference tree (recommended):

```
src/  
  main.cpp  
  core/  
    config.h  
    types.h  
    time_system.h / time_system.cpp  
    world.h / world.cpp  
  ecs/  
    components.h  
    entity.h  
    registry.h / registry.cpp  
  physics/  
    physics_components.h  
    physics_system.h / physics_system.cpp  
    collision_system.h / collision_system.cpp  
  anm_bridge/  
    perception.h / perception.cpp
```

Rule of thumb: if a file is required for simulation determinism, it must live in core, ecs, or physics. Anything UI or GPU-specific stays outside the kernel.

5. Core loop: time, determinism, 60 Hz physics

The engine tick takes real delta time (wall clock) and converts it into N fixed simulation steps. Each step advances physics by `FIXED_DT`, typically 1/60.

```
// Pseudocode
accumulator += real_dt;
while (accumulator >= FIXED_DT && steps < MAX_STEPS_PER_FRAME) {
    physics_step(FIXED_DT);
    accumulator -= FIXED_DT;
}
```

The `MAX_STEPS_PER_FRAME` budget prevents a spiral of death if the app stalls. For determinism tests, feed the same sequence of `real_dt` values (or drive fixed steps directly).

6. ECS: entities and components

ECS keeps simulation data compact and cache-friendly. The initial NE foundation uses a simple Registry that can: create entities and attach components like Transform, Velocity, Mass, and Force.

Component rules: no pointers to other components, no hidden allocations during hot loops, and prefer SoA layouts.

7. Physics: forces, integration, collisions

Physics step ordering (current): (1) apply forces (e.g., gravity), (2) integrate (semi-implicit Euler to start), (3) resolve collisions (ground plane baseline).

Collision v0.1 focuses on stability. Ground plane collision is the first deterministic contact solver. Next: sphere-sphere, AABB, and broadphase.

8. Rendering: interfaces first, implementations later

Rendering is intentionally not required for NE v0.1. Define a `Renderer` interface that takes a read-only view of the Registry and a Camera, then implement backends later (Metal for Apple devices).

Ray tracing can be introduced once the raster baseline is correct and the engine has stable spatial acceleration structures.

9. ANM bridge: how ANM will interact with NE

ANM should be attached after the kernel is stable. The bridge is a thin API surface that provides:

- Perception: query world state snapshots (positions, velocities, contacts, environment).
- Actions: apply forces/impulses, spawn entities, set goals, schedule scenarios.
- Safety: clamp action magnitudes and validate constraints to avoid destabilizing physics.

10. Build and run (macOS, clang++)

For the early phase, compiling from the terminal is the most reliable path. Example command (adapt paths to your repo):

```
clang++ -std=c++20 -O3 \  
src/main.cpp \  
src/core/time_system.cpp src/core/world.cpp \  
src/ecs/registry.cpp \  
src/physics/physics_system.cpp src/physics/collision_system.cpp \  
src/anm_bridge/perception.cpp \  
-o nebula  
  
./nebula
```

If you later switch to CMake, keep targets small: one static library for nebula_core and one executable for scenarios/tests.

11. Cursor workflow tips

Cursor works best when the project has tight boundaries. Keep a top-level README with: build command, module ownership, and coding rules (determinism, no allocations in hot loops, unit conventions).

- Create a short prompt file: 'NE rules + module map' and pin it in Cursor.
- Ask Cursor to modify one module at a time (physics, ecs, time) to avoid cross-file chaos.
- Always add a tiny test or demo scenario after each change (gravity drop, collision bounce).

12. Roadmap (ant to supernova)

Milestone	What ships	Proof it works
M0 (now)	Headless kernel: ECS + fixed timestep + gravity	Controlled collision determinism test
M1	Broadphase + basic shapes (sphere/AABB) + stable contacts	Simple contact test, no jitter explosion
M2	Constraints + joints + friction baseline	Pendulum / chain scenario
M3	World scaling + unit system + spatial partitions	Ant scene and city block scene share code
M4	Renderer interface + debug draw + camera	Visual sanity: trajectories and contacts
M5	Metal renderer backend + PBR baseline	Stable 60 FPS on Apple silicon
M6	Ray tracing path (hybrid) + denoise	Realistic lighting tuned to reality
M7	Astro module (n-body, gas, radiation models)	Planet orbit and supernova sandbox

Next action: keep physics + determinism locked, then introduce rendering as a separate optional module. Only after M3-M4 should ANM be connected for interactive control.